

Plan d'apprentissage RAG local

De zéro à un pipeline RAG d'entreprise opérationnel

Validé en lab sur matériel CPU sans GPU

Architecture : LABO-G9 (Ollama, Windows) + VM-RAG-LAB (Ubuntu 26.04, Docker Compose)

Dolt4Everyone | Septembre 2026

Ce plan documente l'apprentissage progressif d'un pipeline RAG complet, tel qu'il a été construit et validé en lab. La progression est linéaire : chaque phase s'appuie sur la précédente. Durée estimée : 18 à 22 semaines à raison de 4 à 6 heures par semaine. Les phases 1 à 12 sont entièrement validées en lab sur CPU. Les phases 13 et 14 nécessitent un tenant Microsoft 365 actif et sont en cours de validation.

1. Architecture cible et prérequis

1.1 Environnement de travail

L'apprentissage se fait dans une configuration qui reproduit ce qu'un client PME aurait en production : Ollama sur le serveur Windows, pipeline RAG dans une VM Linux séparée, communication réseau entre les deux.

Composant	Emplacement	Rôle
Ollama (inférence LLM)	LABO-G9, Windows, port 11434	Serveur LLM : génération de texte et embeddings
nomic-embed-text	Ollama sur LABO-G9	Modèle d'embedding (vectorisation des chunks)
qwen2.5:14b	Ollama sur LABO-G9	LLM principal de la stack validée
qwen3:4b	Ollama sur LABO-G9	Modèle juge pour le groundedness check
VM-RAG-LAB	VMware Workstation sur LABO-G9	Ubuntu Server 26.04 LTS, stack Docker Compose
Qdrant	Conteneur Docker dans VM-RAG-LAB	Vector store : embeddings et métadonnées ACL
RAG API FastAPI	Conteneur Docker dans VM-RAG-LAB	API de retrieval, génération et journalisation nLPD
Open WebUI	Conteneur Docker dans VM-RAG-LAB	Interface utilisateur, authentification LDAP AD
n8n	Conteneur Docker dans VM-RAG-LAB	Orchestration : sync corpus, rappels rotation

1.2 Réseau VMware

La VM-RAG-LAB doit atteindre Ollama sur le port 11434 de l'hôte. Deux options :

- VMnet8 (NAT) : la VM accède à l'hôte via l'IP de passerelle NAT. Plus simple, recommandé pour débiter.
- Réseau lab dédié : réseau isolé avec accès aux ressources AD, SMB et DNS. Recommandé pour les phases 5 à 9.

Vérification avant de commencer : depuis la VM, `curl http://<IP-HOTE>:11434/api/tags` doit retourner la liste des modèles Ollama. Si la réponse est vide, vérifier que `OLLAMA_HOST=0.0.0.0:11434` est défini sur LABO-G9.

1.3 Sizing de la VM

Ressource	Minimum	Recommandé	Remarque
vCPU	4	6	1 processeur, 6 coeurs dans VMware (évite les problèmes NUMA)
RAM	16 Go	16 Go	Stack Docker complète : 8 à 10 Go en charge
Disque	80 Go	100 Go	Type NVMe recommandé dans VMware Workstation
Réseau	NAT	Réseau lab dédié	Requis pour accès AD, SMB, DNS (phases 5 à 9)

1.4 Première action obligatoire

Avant toute chose, télécharger les deux modèles sur LABO-G9 :

```
ollama pull nomic-embed-text # modèle d'embedding, ~274 Mo, CPU  
uniquement  
ollama pull qwen2.5:14b      # LLM principal, ~9 Go  
ollama pull qwen3:4b        # modèle juge groundedness check, ~2,5 Go
```

2. Phases d'apprentissage

Le plan compte 12 phases validées en lab, complétées par 2 phases optionnelles nécessitant un tenant Microsoft 365 actif.

Phase	Sujet	Durée
1	Environnement VM et premier modèle	1 semaine (4-5h)
2	Premier pipeline RAG custom Python	1 semaine (4-5h)
3	Parsing des documents et stratégie de chunking	1-2 semaines (6-8h)
4	Qdrant : persistance, métadonnées et snapshot	1 semaine (4-5h)
5	Prompt strict, citations et groundedness check	1-2 semaines (6-8h)
6	Stack Docker Compose complète	1 semaine (4-5h)
7	Permissions NTFS : filtrage par identité AD	2-3 semaines (10-15h)
8	Open WebUI : interface et authentification LDAP	1 semaine (4-5h)
9	Sécurité : injection, durcissement, journalisation nLPD	1-2 semaines (6-8h)
10	Synchronisation et monitoring via n8n	1 semaine (4-5h)
11	Onyx CE : évaluation comparative	1 semaine (4-5h)
12	Projet de synthèse sur corpus réel	2-3 semaines (10-15h)
13 (optionnelle)	Connecteur SharePoint Online et permissions Entra ID	2-3 semaines
14 (optionnelle)	Pipeline résumé de réunions Teams	1-2 semaines

Phase 1 : Environnement VM et premier modèle 1 semaine · 4-5h

Objectif : avoir une VM opérationnelle, Ollama accessible depuis la VM, comprendre la différence entre un modèle d'embedding et un modèle de langage. Initialiser le dépôt Git qui suivra tous les scripts produits.

Créer la VM-RAG-LAB

- Ubuntu Server 26.04 LTS, 6 vCPU (1 processeur, 6 coeurs), 16 Go RAM, 80 Go NVMe dans VMware Workstation.
- Réseau : VMnet8 (NAT) pour commencer.
- Lors de l'installation : laisser LVM activé, activer OpenSSH, configurer le réseau en statique si la VM rejoint un réseau lab existant.
- Étendre le volume LVM après installation (Ubuntu n'alloue que 50% du VG par défaut) :

```
lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
resize2fs /dev/mapper/ubuntu--vg-ubuntu--lv
```
- Installer Python 3.11, pip, venv, Docker et Docker Compose.
- Initialiser un dépôt Git : `git init ~/rag-lab && cd ~/rag-lab`. Committer après chaque phase.

Valider la connectivité Ollama

```
curl http://<IP-HOTE-G9>:11434/api/tags
```

La réponse doit lister les modèles disponibles. Si Ollama n'écoute que sur 127.0.0.1, définir `OLLAMA_HOST=0.0.0.0:11434` sur LABO-G9 (variable d'environnement utilisateur Windows).

Premier appel LLM depuis la VM

```
curl http://<IP-HOTE>:11434/api/generate \
-d '{"model":"qwen2.5:14b","prompt":"Bonjour","stream":false}'
```

Observer le temps de réponse sur CPU. C'est lent, c'est normal : cette lenteur sera la référence pour objectiver le gain GPU plus tard.

Ce que vous apprenez ici : Différence entre un modèle de langage (génère du texte) et un modèle d'embedding (transforme du texte en vecteur). Pourquoi la RAM est le facteur limitant sur CPU. Pourquoi on a besoin des deux types dans un pipeline RAG.

Phase 2 : Premier pipeline RAG custom Python 1 semaine · 4-5h

Objectif : indexer un dossier de documents et poser une première question à laquelle le LLM répond à partir des documents, pas de sa mémoire d'entraînement. Observer les limites du chunking par défaut : c'est le point de départ de la phase 3.

Ce plan choisit un pipeline Python custom plutôt que LlamaIndex. La raison est pratique : LlamaIndex ajoute une couche d'abstraction utile pour l'apprentissage des concepts, mais qui masque les comportements réels du chunking et du retrieval. Le pipeline custom donne un contrôle total sur chaque étape, ce qui est indispensable pour les phases 5 à 9.

Installation de l'environnement Python

```
python3 -m venv .venv && source .venv/bin/activate
pip install qdrant-client httpx python-docx pdfplumber python-pptx
```

Corpus de test recommandé

- Des documents Word (.docx) avec tableaux et listes : ce sont les cas qui cassent le chunking fixe.
- Un PDF textuel et un PDF scanné (image) : pour observer la différence de qualité d'extraction.
- Quelques fichiers .md de documentation technique : pour tester le comportement sur les sections courtes.

Inclure au minimum un fichier de chaque type. Les cas qui échouent en phase 2 seront les cas de test de la phase 3.

Premier script d'indexation

```
# Extraction de texte par type de fichier
# .docx : python-docx (paragraphes et cellules de tableaux)
# .pdf : pdfplumber (page par page, avec détection des scans)
# .pptx : python-pptx (texte de chaque diapositive)
# .txt, .md : split sur double saut de ligne (paragraphes)
```

```
# Embedding via Ollama (nomic-embed-text)
# Stockage dans Qdrant avec métadonnées minimales : source, hash MD5
```

Ce que vous apprenez ici : *Le chunking automatique à taille fixe produit souvent des chunks qui coupent un paragraphe ou un tableau en milieu de phrase. La qualité du chunking détermine la qualité maximale atteignable par le RAG, avant même le prompt.*

Phase 3 : Parsing des documents et stratégie de chunking 1-2 semaines · 6-8h

Objectif : comprendre pourquoi le chunking est la couche qui a le plus d'effet sur la qualité du RAG. Comparer deux approches sur le même corpus et mesurer l'impact sur les questions qui avaient échoué en phase 2.

La stratégie retenue : blocs de paragraphes

Le plan original proposait LlamaIndex SemanticSplitterNodeParser. La validation terrain a montré qu'une approche plus simple est plus robuste : découper sur les doubles sauts de ligne (blocs de paragraphes naturels), avec un seuil minimal de mots pour éviter d'indexer des fragments sans sens.

```
MIN_CHUNK_WORDS = 8      # seuil minimal, configurable via .env
CHUNK_SIZE       = 150    # taille cible en mots
CHUNK_OVERLAP    = 20     # recouvrement entre chunks
```

Le seuil de 8 mots est critique pour les fichiers .md à sections courtes : avec un seuil de 15 (valeur naïve), la majorité des sections courtes sont silencieusement ignorées et le fichier entier produit 0 chunk.

Cas particuliers validés en lab

- PDF scanné sans couche texte : pdfplumber retourne une page vide. Le fichier part en quarantaine avec un avertissement, pas en erreur silencieuse.
- Fichier .md avec sections courtes (titre + une phrase) : le seuil MIN_CHUNK_WORDS=8 permet de les capturer.
- Tableau Word avec cellules fusionnées : extraction cellule par cellule via python-docx, chaque ligne devient un bloc.

Mesurer l'impact

Sur les 10 questions qui avaient échoué en phase 2 : indexer le corpus avec le nouveau chunking, poser les mêmes questions, noter le nombre de bonnes réponses. La progression est la mesure la plus parlante de ce que le chunking apporte.

Ce que vous apprenez ici : Pourquoi 'chunk = paragraphe' est la règle de base, pas le découpage à taille fixe. Pourquoi les tableaux et les listes sont les cas les plus difficiles. Pourquoi un seuil minimal de mots est nécessaire pour les documents à sections courtes.

Phase 4 : Qdrant : persistance, métadonnées et snapshot 1 semaine · 4-5h

Objectif : déployer Qdrant en conteneur Docker. L'index survit aux redémarrages. Prendre un premier snapshot avant d'aller plus loin.

Lancer Qdrant avec Docker Compose

```
services:
  qdrant:
    image: qdrant/qdrant:latest
    ports: ["6333:6333"]
    volumes: ["/qdrant_data:/qdrant/storage"]
    restart: unless-stopped
```

Métadonnées à stocker pour chaque chunk

Champ	Contenu	Usage
source	Chemin relatif du fichier sur le partage SMB	Citation dans la réponse
hash_md5	Hash MD5 du fichier au moment de l'indexation	Détection des modifications
org_name	Organisation déduite de la structure des dossiers	Navigation et contexte
autorises	Liste des groupes AD autorisés (ACL NTFS)	Filtrage d'accès (phase 7)
interdits	Liste des groupes AD refusés (DENY explicites)	Filtrage d'accès (phase 7)
text	Contenu du chunk	Injection dans le prompt

Les champs autorises et interdits sont vides à ce stade. Ils seront peuplés par `acl_resolver.py` en phase 7. Les initialiser vides dès maintenant évite de restructurer le schéma plus tard.

Prendre un snapshot Qdrant

```
curl -X POST http://localhost:6333/collections/documents/snapshots
```

Télécharger le snapshot et le conserver hors du conteneur. C'est la sauvegarde de référence pour toutes les phases suivantes.

Ce que vous apprenez ici : Architecture d'un vector store de production. Différence entre recherche vectorielle (similarité sémantique) et filtre de métadonnées (valeur exacte). Les deux seront combinés pour le filtrage par permissions en phase 7.

Phase 5 : Prompt strict, citations et groundedness check 1-2 semaines · 6-8h

Objectif : construire les guardrails qui rendent le pipeline fiable en production. C'est la phase la plus importante pour la conformité nLPD.

Prompt système strict

```
Tu es un assistant documentaire. Tu réponds UNIQUEMENT à partir des documents fournis dans le contexte ci-dessous.

Si la réponse n'est pas dans les documents fournis, réponds exactement :
'Cette information ne figure pas dans les documents disponibles.'

Tu ne complètes jamais avec tes connaissances générales.
Chaque affirmation doit être directement tirée d'un document du contexte.
Cite le nom exact du fichier source entre crochets : [nom_du_fichier.docx]
Ne jamais écrire [Document X] ou numéroter les sources.

Ne résume jamais un document si son contenu ne répond pas directement à la question posée. Un document hors sujet doit être ignoré, pas résumé.
/no_think
```

Contrôles déterministes (sans coût de calcul)

- Contrôle 1 : aucun chunk récupéré : la réponse est bloquée immédiatement.
- Contrôle 2 : réponse de refus standard (moins de 200 caractères contenant la phrase de refus) : la vérification passe.
- Contrôle 3 : citations inventées : chaque citation entre crochets est comparée aux sources Qdrant réelles.
- Contrôle 4 : réponse longue sans citation : toute réponse de plus de 200 caractères sans citation est bloquée.

Groundedness check : juge LLM

Un second appel Ollama post-génération avec qwen3:4b vérifie affirmation par affirmation. Le juge utilise le paramètre keep_alive pour rester chargé en mémoire entre deux questions.

```
JUDGE_KEEP_ALIVE=2h # rétention en mémoire après chaque appel
```

Instructions du juge (ajoutées en phase 5, validées en lab) :

- Une affirmation est NON sourcée si elle attribue une fonctionnalité au mauvais sujet.
- Si la source documente un outil construit autour d'un produit, la réponse ne peut pas présenter cet outil comme une fonctionnalité native du produit.

Limite sur CPU : qwen3:4b détecte les hallucinations grossières (chiffres, dates, noms inventés) mais peut rater les mauvaises attributions subtiles. Une validation fiable des attributions nécessite un modèle juge plus grand, disponible une fois le GPU installé. Par ailleurs, une injection en texte libre dans un document indexé peut tromper le modèle sur CPU : le juge détecte la réponse non ancrée (ancee: false) mais le blocage automatique sur /v1 est prévu après l'installation du GPU.

warmup_judge : maintenir le juge chargé

A chaque requête RAG, un ping de 0 token est envoyé au juge avant le retrieval. Ollama recharge le modèle pendant la génération (2 à 5 minutes sur CPU), de sorte que le juge est prêt quand on en a besoin.

```
# Ping du juge : /api/generate avec prompt vide et keep_alive  
# Appelé au début de chaque requête, avant search_qdrant()
```

Ce que vous apprenez ici : Comment le prompt contraint la génération. Pourquoi la température doit être basse (0.0 à 0.2) pour les tâches factuelles. La distinction entre 'réponse fluide' et 'réponse correcte'. Ce que mesure le faithfulness vs la qualité perçue.

Phase 6 : Stack Docker Compose complète 1 semaine · 4-5h

Objectif : déployer la stack complète via Docker Compose. C'est la couche qui transforme des scripts en service opérationnel.

Services déployés

Service	Port	Rôle	Image
Qdrant	6333	Vector store	qdrant/qdrant:latest
n8n	5678	Orchestration pipelines	n8nio/n8n:latest
RAG API	8080	API FastAPI : retrieval, génération, sync	Build local (api/)
Open WebUI	3001	Interface utilisateur, auth LDAP	ghcr.io/open-webui/open-webui:main

Variables d'environnement clés

```
LLM_BASE_URL=http://<IP-HOTE-OLLAMA>:11434
LLM_MODEL=qwen2.5:14b
JUDGE_MODEL=qwen3:4b
JUDGE_KEEP_ALIVE=2h
EMBED_MODEL=nomic-embed-text
QDRANT_HOST=http://qdrant:6333
MIN_CHUNK_WORDS=8
TOP_K=20
JWT_EXPIRES_IN=4h
```

Structure des répertoires

```
~/rag-stack/
├── .env                <- variables sensibles, ne pas committer
├── docker-compose.yml
├── qdrant_data/        <- données Qdrant (persistantes)
├── n8n_data/           <- workflows n8n (persistants)
├── openwebui_data/     <- données Open WebUI (persistantes)
└── api/
    ├── Dockerfile
    ├── requirements.txt
    ├── main.py         <- RAG API complète
    └── auth.py         <- résolution LDAP

/root/rag-pipeline/    <- scripts d'indexation (montés en lecture
seule)
├── indexer.py
└── acl_resolver.py
```

Ce que vous apprendrez ici : Comment un pipeline RAG devient un service. Gestion de la concurrence à faible échelle sur CPU. Structure d'une réponse API avec citations et score de confiance.

Phase 7 : Permissions NTFS : filtrage par identité AD 2-3 semaines · 10-15h

Objectif : propager les permissions AD/NTFS jusqu'aux chunks Qdrant et filtrer les résultats selon l'identité de l'utilisateur. C'est la phase la plus différenciante vis-à-vis d'un intégrateur standard.

Préparer avant de démarrer : deux comptes AD avec des droits différents sur au moins deux fichiers du corpus. Les groupes imbriqués et les DENY prendront plus de temps que prévu. Prévoir 10 à 15 heures, pas 6 à 8.

Lecture des ACL NTFS depuis Linux

```
# Monter le partage SMB (persistant via fstab)
//<FILESERVER>/PartageDocuments /mnt/corpus-root cifs \
credentials=/etc/smbcredentials/svc-rag,vers=3.1.1 0 0

# Lire les ACL via smbcacls
smbcacls //<FILESERVER>/PartageDocuments fichier.docx -U svc-rag
```

Parser la sortie pour extraire les groupes autorisés (ALLOW) et refusés (DENY) pour chaque fichier. Stocker dans les champs autorises[] et interdits[] de chaque chunk Qdrant.

Résolution LDAP récursive

Un utilisateur peut être membre d'un groupe qui est lui-même membre d'un autre groupe. La résolution doit être récursive sur l'attribut memberOf de l'AD.

```
# Résolution récursive via auth.py
# Validé en lab : 3 niveaux d'imbrication résolus correctement
# Cache TTL = 5 minutes pour éviter un appel LDAP à chaque requête
```

Filtrage à la requête

```
# Filtre Qdrant : intersection des groupes AD de l'utilisateur
# avec autorises[], exclusion si intersection avec interdits[]
# DENY explicite prioritaire sur ALLOW : cas le plus souvent raté
```

Tests obligatoires

- Cloisonnement positif : un utilisateur membre de GRP-Clients voit les documents CLIENTS.
- Cloisonnement négatif : ce même utilisateur ne voit pas les documents RH.
- DENY explicite : un utilisateur avec DENY nominatif sur un fichier ne le voit pas, même si son groupe est dans autorises[].
- Groupes imbriqués : un utilisateur dans GRP-Clients-Niveau2 (membre de GRP-Clients) voit les documents CLIENTS.

Le cas DENY explicite est le cas que la majorité des implémentations ratent. Le tester explicitement avant de passer à la phase suivante.

Ce que vous apprenez ici : Pourquoi les DENY sont prioritaires sur les ALLOW dans Windows. La différence entre indexer avec un compte admin et servir avec un compte utilisateur. Pourquoi la resynchronisation des ACL est un job distinct de la resynchronisation des contenus.

Phase 8 : Open WebUI : interface et authentification LDAP 1 semaine · 4-5h

Objectif : déployer Open WebUI comme interface utilisateur finale, avec authentification LDAP Active Directory et transmission de l'identité à la RAG API pour le filtrage documentaire.

Configuration LDAP dans Open WebUI

Paramètre	Valeur	Note
Port	636	LDAPS obligatoire (port 389 refusé par les DC récents)
Attribut email	userPrincipalName	mail souvent non renseigné dans l'AD
Attribut username	sAMAccountName	
Base de recherche	DC=domaine,DC=ch	Pas OU=UTILISATEURS : certains comptes sont dans CN=Users
Validate Certificate	Activé en prod	Exige le bundle ad-chain.pem exporté depuis le DC

Transmission de l'identité à la RAG API

La variable `ENABLE_FORWARD_USER_INFO_HEADERS=true` est indispensable. Sans elle, Open WebUI traite la RAG API comme un simple endpoint OpenAI et ne transmet pas l'identité. Les headers transmis sont :

```
x-openwebui-user-email    <- utilisé par auth.py pour la résolution LDAP
x-openwebui-user-name
x-openwebui-user-id
x-openwebui-user-role
```

Validation de bout en bout

- Connecter Open WebUI à la RAG API via Settings → Admin → Connections → API compatibles OpenAI.
- Tester avec deux comptes AD aux droits différents : vérifier que chacun ne voit que son périmètre.
- Vérifier dans les logs rag-api que l'email et les groupes AD sont correctement résolus.

Ce que vous apprenez ici : Comment Open WebUI transmet l'identité de l'utilisateur à une API tierce. Pourquoi l'attribut `userPrincipalName` est préférable à `mail` dans un AD Windows. La chaîne complète : navigateur → Open WebUI → RAG API → Qdrant → Ollama.

Phase 9 : Sécurité : injection, durcissement, journalisation nLPD 1-2 semaines · 6-8h

Objectif : tester les vecteurs d'attaque du pipeline RAG et mettre en place les contre-mesures. C'est la phase la plus importante pour la mise en production.

La sécurité du pipeline RAG est l'angle mort de la quasi-totalité des offres d'intégrateurs. La tester soi-même est le seul moyen d'en parler avec autorité.

Test 1 : injection de prompt via document indexé

Créer un fichier PDF contenant du texte invisible (blanc sur fond blanc) avec une instruction hostile. Indexer ce fichier dans le corpus. Poser une question normale. Observer si le modèle suit l'instruction injectée.

Mitigation : séparation structurelle entre le prompt système (fixe) et les chunks (zone 'données'). Vérifier que l'injection échoue après mitigation.

Test 2 : accès sans authentification

Interroger l'endpoint /query via curl sans token. Si le service répond, le pipeline est exposé sans contrôle d'accès. Implémenter un Bearer token sur FastAPI et vérifier le rejet des requêtes non authentifiées.

Test 3 : svc-rag désactivé

Piège validé en lab : si svc-rag est désactivé alors que la synchronisation n8n tourne, les chunks sont réindexés sans ACL. Tous les utilisateurs sont bloqués (sources_accessed vide). Ne jamais désactiver svc-rag sans couper la synchronisation n8n au préalable.

Journalisation nLPD

Logger chaque requête dans un fichier JSONL avec les champs suivants :

```
{
  "timestamp": "2026-09-15T09:20:48Z",
  "user_id": "utilisateur@domaine.ch",
  "question_hash": "a394957914563e32",
  "sources_accessed": ["CLIENTS/contrat.docx", ...],
  "ancree": true,
  "verification": "effectuee"
}
```

La question est hashée, pas stockée en clair. On peut démontrer qu'une question a été posée et quels documents ont été consultés, sans exposer le contenu des échanges.

Pare-feu UFW

```
ufw default deny incoming
ufw allow 22/tcp
ufw allow from <SUBNET>/24 to any port 8080 # RAG API
ufw allow from <SUBNET>/24 to any port 3001 # Open WebUI
ufw allow from <SUBNET>/24 to any port 5678 # n8n
ufw allow from <SUBNET>/24 to any port 6333 # Qdrant
```

TLS LDAP

Exporter le bundle de certificats du DC (Sub CA + Root CA) et le monter dans le conteneur rag-api. Configurer LDAP_CA_CERT=/etc/ssl/certs/ad-chain.pem et activer CERT_REQUIRED. Toujours utiliser le nom DNS du DC, jamais l'IP, pour que la validation du certificat fonctionne.

Ce que vous apprenez ici : Comment un PDF anodin peut compromettre un pipeline RAG non protégé. Pourquoi la séparation structurelle instructions/données est la mitigation principale. Pourquoi la journalisation est une exigence nLPD, pas une option. Le piège de la synchronisation avec svc-rag désactivé.

Phase 10 : Synchronisation et monitoring via n8n 1 semaine · 4-5h

Objectif : automatiser la synchronisation du corpus et la résolution des ACL. Un RAG non maintenu se dégrade sur trois vecteurs : contenu périmé, permissions obsolètes, dérive du modèle.

Synchronisation du corpus via /admin/sync

L'endpoint /admin/sync de la RAG API lance successivement indexer.py puis acl_resolver.py dans le conteneur. Les scripts ont accès au partage SMB monté sur l'hôte.

```
curl -X POST http://localhost:8080/admin/sync \
-H "Authorization: Bearer <ADMIN_TOKEN>"
```

Pipeline n8n : synchronisation horaire

- Schedule Trigger : toutes les heures.
- HTTP Request : POST /admin/sync avec le token admin.
- IF Node : si success=false, envoyer une alerte email.

La synchronisation ACL est un job distinct de la synchronisation du contenu. Un fichier peut ne pas avoir changé mais ses permissions ACL ont été modifiées. Les deux jobs sont lancés en séquence par /admin/sync.

Pipeline n8n : rappel rotation svc-rag

- Schedule Trigger : le premier lundi du mois.
- Email : rappel de rotation du mot de passe svc-rag.

La rotation se fait manuellement : nouveau mot de passe AD, mise à jour du .env, rebuild rag-api.

Ce que vous apprenez ici : Les trois vecteurs de dégradation d'un RAG : contenu périmé, permissions obsolètes, dérive du modèle. Pourquoi la synchronisation ACL doit être un job séparé de la synchronisation des contenus. Le piège de svc-rag désactivé pendant une synchronisation.

Phase 11 : Onyx CE : évaluation comparative 1 semaine · 4-5h

Objectif : déployer Onyx Community Edition et comprendre ce qu'une plateforme packagée apporte par rapport au pipeline custom construit dans les phases précédentes. Identifier ses limites.

Déploiement Onyx CE

```
git clone https://github.com/onyx-dot-app/onyx.git
cd onyx/deployment/docker_compose
# Modifier .env pour pointer vers Ollama sur LABO-G9
docker compose -f docker-compose.dev.yml up -d
```

Onyx est accessible sur le port 3000. OpenSearch consomme 2 Go de RAM par défaut : limiter à 512 Mo pour un lab. La variable OPENSEARCH_JAVA_OPTS n'est pas lue depuis .env par le conteneur OpenSearch d'Onyx : elle doit être ajoutée dans la section environment du service index de docker-compose.dev.yml (OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m).

Comparaison sur le même corpus

Critère	Pipeline custom (phases 1-10)	Onyx CE
Qualité des réponses	Mesurée (faithfulness)	A mesurer sur le même jeu
Cloisonnement NTFS	Validé (autorises[], DENY)	Non natif
Groundedness check	qwen3:4b, règles attribution	Non inclus
Journalisation nLPD	JSONL structuré, hash question	Non inclus
Maintenance	Scripts Python, Docker Compose	Stack complexe (OpenSearch, Redis, Celery)
Formats indexés	.docx, .pdf, .pptx, .txt, .md	40+ connecteurs

Ce que vous apprenez ici : Valeur et limites d'une plateforme packagée. Ce que représente la maintenance d'une stack open-source. Comment évaluer objectivement une offre d'intégrateur qui présente Onyx comme solution clé en main.

Phase 12 : Projet de synthèse sur corpus réel 2-3 semaines · 10-15h

Objectif : construire un RAG complet sur un cas d'usage réel. C'est la phase qui transforme l'apprentissage en compétence démontrable.

Cas d'usage recommandé

Base de connaissances technique interne : les guides, procédures et documentation sur un domaine maîtrisé constituent un corpus de qualité. Les bonnes réponses sont connues d'avance, ce qui permet d'évaluer le système sans ambiguïté.

Livrable du projet

- Pipeline complet dockerisé : Qdrant + RAG API FastAPI + Open WebUI + n8n + synchronisation + ACL NTFS + journalisation nLPD.
- Stratégie de chunking documentée et mesurée sur le corpus réel (MIN_CHUNK_WORDS justifié).
- Rapport de déploiement : choix techniques, modèles utilisés, résultats faithfulness mesurés.
- Jeu de 30 questions de test avec réponses attendues et résultats obtenus.
- Documentation de maintenance : synchronisation du corpus, surveillance de la dérive, procédure de restauration Qdrant.
- Test de sécurité documenté : injection de prompt testée et mitigée, authentification vérifiée, journalisation validée. Script deploy.sh (beta) : déploiement automatisé de la stack, publié dans scripts/stack-ia-locale/ sur GitHub.

Transition vers GPU

Une fois ce projet terminé, la migration vers un GPU ne demande que deux modifications : pointer Ollama vers une instance GPU ou déployer vLLM à la place, et ajuster JUDGE_MODEL=qwen2.5:14b (le modèle principal est déjà en VRAM, le réutiliser comme juge ne coûte pas de VRAM supplémentaire). Le pipeline Python est identique.

```
# Sur GPU : le modèle principal passe à Qwen3-30B-A3B. #
JUDGE_MODEL=qwen3:4b reste le juge (distinct du modèle principal). #
JUDGE_KEEP_ALIVE=-1
# Ollama : OLLAMA_KEEP_ALIVE=-1 (variable d'environnement utilisateur)
```

Les benchmarks CPU constituent une référence pour objectiver le gain GPU. Les mesurer avant la migration.

Critère de réussite du plan complet : déployer, configurer, sécuriser, évaluer et maintenir un pipeline RAG complet, expliquer chaque choix technique à un décideur non-technicien, et identifier les failles d'une offre d'intégrateur. Ce plan couvre les trois.

3. Phases optionnelles : connecteurs Microsoft 365

Ces deux phases nécessitent un tenant Microsoft 365 actif et des droits admin Entra ID. Elles sont optionnelles parce que ce prérequis n'est pas garanti dans un contexte de lab personnel.

Note de souveraineté nLPD : si le tenant MS 365 n'est pas hébergé en Suisse, les requêtes d'indexation vers SharePoint Online transitent hors juridiction suisse, même si le LLM tourne en local. La souveraineté des données dépend du datacenter du tenant, pas du modèle d'inférence.

Phase 13 : Connecteur SharePoint Online et permissions Entra ID 2-3 semaines

· Prérequis : tenant MS 365, droits admin Entra ID

Objectif : connecter le pipeline RAG aux documents SharePoint Online avec respect des permissions Entra ID, via Graph API.

App Registration dans Entra ID

- Permission à accorder : Sites.Selected (application, pas déléguée). Ne jamais utiliser Sites.Read.All.
- Accorder l'accès à un site SharePoint spécifique via PowerShell (Grant-PnPazureADAppSitePermission).
-

Propagation des permissions Entra ID vers Qdrant

Même principe que les SIDs NTFS en phase 7, mais avec les Object IDs des groupes Entra ID. Graph API expose les permissions de chaque document via GET /sites/{site-id}/drive/items/{item-id}/permissions.

Fichiers chiffrés Purview

Les fichiers protégés par des labels de sensibilité Microsoft Information Protection sont chiffrés par Azure Rights Management. L'indexeur ne peut pas en lire le contenu directement. La piste envisagée : clé RMS consultée à la volée depuis Azure Key Vault au moment de l'indexation, sans jamais stocker la clé dans le conteneur.

Ce que vous apprenez ici : Différence entre Sites.Read.All (dangereux) et Sites.Selected (correct). Comment les permissions cloud se propagent dans un vector store local. Pourquoi la souveraineté des données dépend du datacenter du tenant, pas du LLM.

Phase 14 : Pipeline résumé de réunions Teams 1-2 semaines · Prérequis : tenant MS 365 avec Teams, droits admin tenant

Objectif : construire le pipeline complet de récupération des transcriptions Teams, de génération du compte-rendu structuré et de dépôt dans le canal Teams concerné.

Prérequis admin

- Activer la transcription automatique dans le centre d'administration Teams (admin.teams.microsoft.com).
- Attendre la propagation de la stratégie (jusqu'à 24h selon le tenant).

Pipeline n8n

- GET /users/{userId}/onlineMeetings : récupérer les réunions du jour.
- GET /me/onlineMeetings/{meetingId}/transcripts/{transcriptId}/content : récupérer la transcription VTT.
- Appel Ollama avec prompt de structuration (participants, décisions, actions, points ouverts).
- POST /teams/{teamId}/channels/{channelId}/messages : déposer le compte-rendu dans le canal Teams.

Permissions Graph API requises

- OnlineMeetings.Read.All : accès aux réunions et à leurs métadonnées.
- OnlineMeetingTranscript.Read.All : accès aux fichiers de transcription VTT.
- ChannelMessage.Send : dépôt du compte-rendu dans le canal Teams.

Ce que vous apprenez ici : Comment Graph API expose les réunions Teams et leurs transcriptions. Pourquoi l'activation admin est un prérequis non négociable. La chaîne complète d'un pipeline MS 365 vers LLM local vers Teams.

Annexe : commandes de référence rapide

Ollama sur LABO-G9

Commande	Description
ollama pull nomic-embed-text	Télécharger le modèle d'embedding (obligatoire en premier)
ollama pull qwen2.5:14b	Télécharger le LLM principal
ollama pull qwen3:4b	Télécharger le modèle juge groundedness
ollama list	Lister les modèles disponibles
ollama ps	Voir les modèles actuellement chargés en mémoire

Docker Compose dans VM-RAG-LAB

Commande	Description
docker compose up -d	Démarrer tous les services en arrière-plan
docker compose down	Arrêter tous les services
docker compose build --no-cache rag-api	Rebuild l'image rag-api (après modification de main.py ou requirements.txt)
docker compose logs -f rag-api	Suivre les logs du service API
docker compose ps	État de tous les services
docker exec rag-api env grep -E 'JUDGE MIN_CHUNK'	Vérifier les variables transmises au conteneur

Qdrant

URL / commande	Description
http://localhost:6333/dashboard	Interface web Qdrant (collections, points, métadonnées)
curl http://localhost:6333/collections	Liste des collections
curl -X POST http://localhost:6333/collections/documents/snapshots	Créer un snapshot de sauvegarde
curl -X POST http://localhost:8080/admin/sync -H 'Authorization: Bearer <TOKEN>'	Lancer la synchronisation du corpus

Ressources

Ressource	URL
Procédures opérationnelles (§1 à §9)	doit4everyone.github.io/ia-locale-pme-suisse/docs/stack-ia-locale/
Scripts Python anonymisés	doit4everyone.github.io/ia-locale-pme-suisse/scripts/stack-ia-locale/
Qdrant documentation	qdrant.tech/documentation
Ollama documentation	ollama.com/docs
Open WebUI documentation	docs.openwebui.com
vLLM documentation (référence DGX Spark)	docs.vllm.ai

Document produit par Doit4Everyone, septembre 2026. Phases 1 à 12 validées en lab sur VM-RAG-LAB (Ubuntu 26.04 LTS), matériel CPU sans GPU. Les versions des bibliothèques évoluent rapidement : toujours vérifier la compatibilité dans la documentation officielle avant installation.